

Chapter 8: Memory Management

- Background
- Swapping
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Segmentation
- Example: The Intel Pentium

Objectives

- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques, including paging and segmentation
- To provide a detailed description of the Intel Pentium, which supports both pure segmentation and segmentation with paging

Background

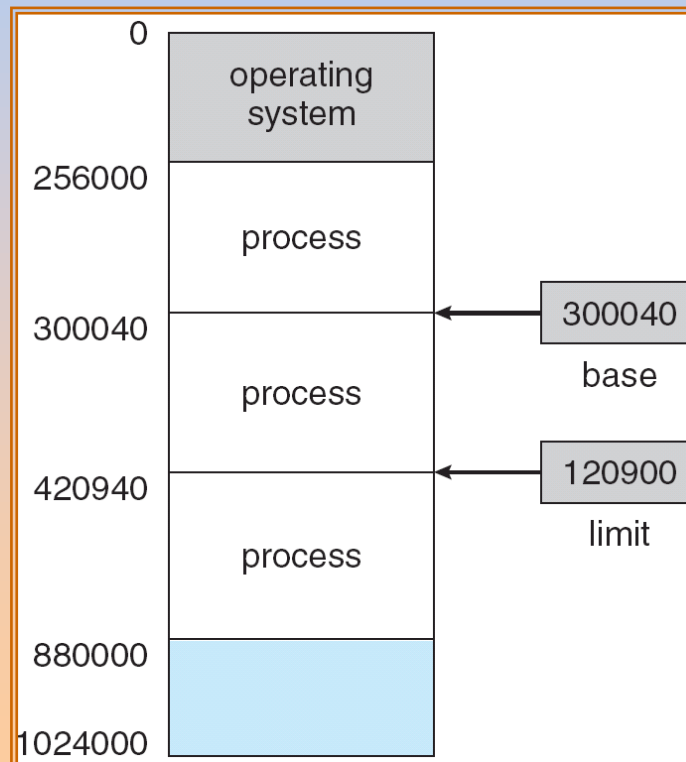
- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Register access in one CPU clock (or less)
- Main memory can take many cycles
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

Background

- *Input queue* – collection of processes on the disk that are waiting to be brought into memory to run the program.
- User programs go through several steps before being run.

Base and Limit Registers

- A pair of **base** and **limit** registers define the logical address space



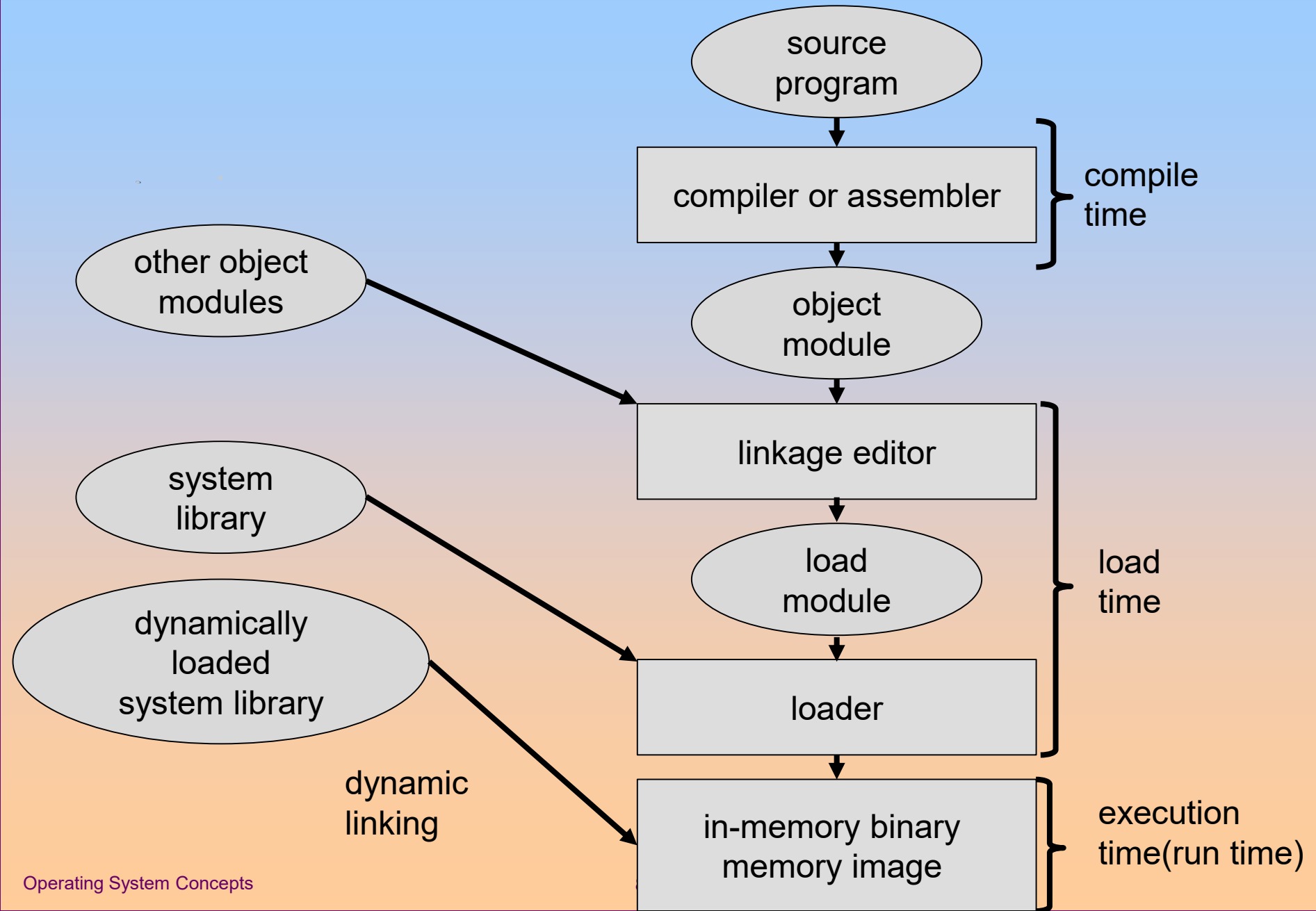
Binding of Instructions and Data to Memory

Address binding (地址联编, 绑定) of instructions and data to memory addresses can happen at three different stages.

- **Compile time:** If memory location known a priori, absolute code can be generated; must recompile code if starting location changes.

- **Load time:** Must generate *relocatable* code if memory location is not known at compile time.
- **Execution time:** Binding delayed until run time if the process can be moved during its execution from one memory segment to another. Need hardware support for address maps (e.g., *base* and *limit registers*).

Multistep Processing of a User Program



Logical vs. Physical Address Space

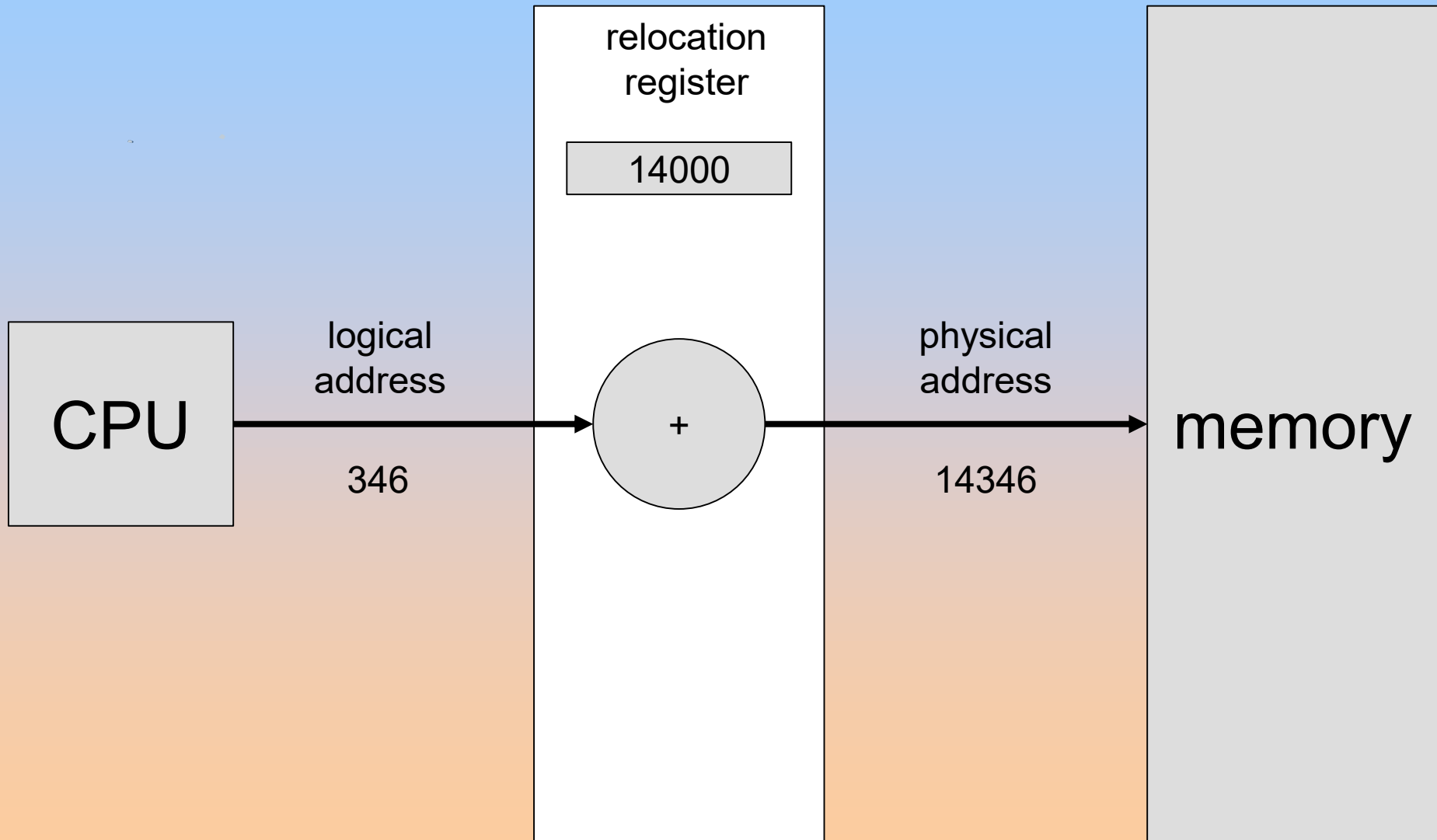
- The concept of a logical *address space* and *physical address space*.
 - ☞ *Logical address* – generated by the CPU; also referred to as *virtual address*.
 - ☞ *Physical address* – address seen by the memory unit.

- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme.

Memory-Management Unit (MMU)

- Hardware device that maps virtual to physical address.
- In MMU scheme, the value in the relocation register is added to every address generated by a user process at the time it is sent to memory.
- The user program deals with *logical* addresses; it never sees the *real* physical addresses.

Dynamic relocation using a relocation register



Dynamic Loading

- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded.
- Useful when large amounts of code are needed to handle infrequently occurring cases.
- No special support from the operating system is required implemented through program design.

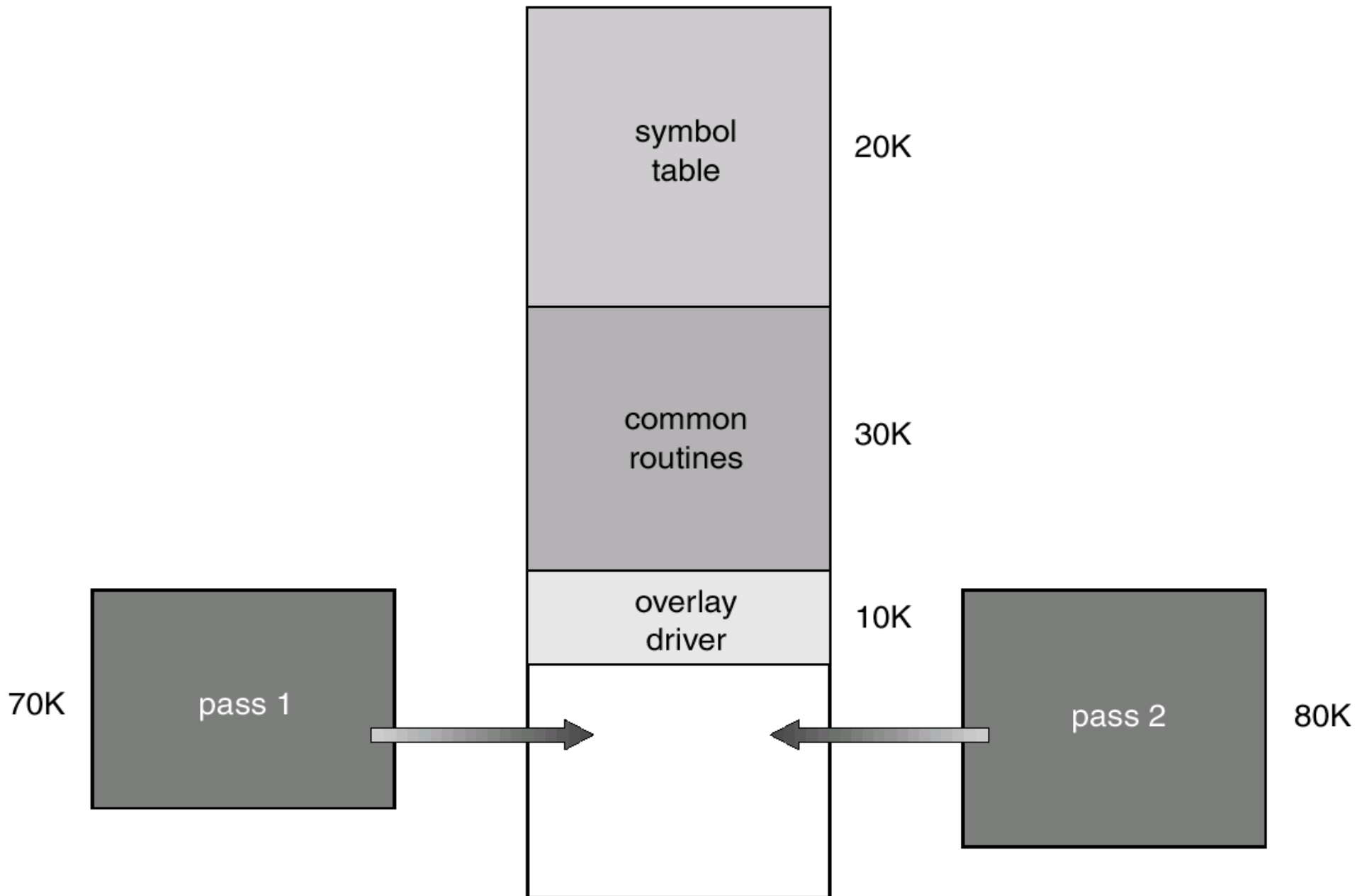
Dynamic Linking

- Linking postponed until execution time.
- Small piece of code, *stub* (存根), used to locate the appropriate memory-resident library routine.
- Stub replaces itself with the address of the routine, and executes the routine.
- Operating system needed to check if routine is in processes' memory address.
- Dynamic linking is particularly useful for libraries.

Overlays

- Keep in memory only those instructions and data that are needed at any given time.
- Needed when process is larger than amount of memory allocated to it.
- Implemented by user, no special support needed from operating system, programming design of overlay structure is complex

Overlays for a Two-Pass Assembler

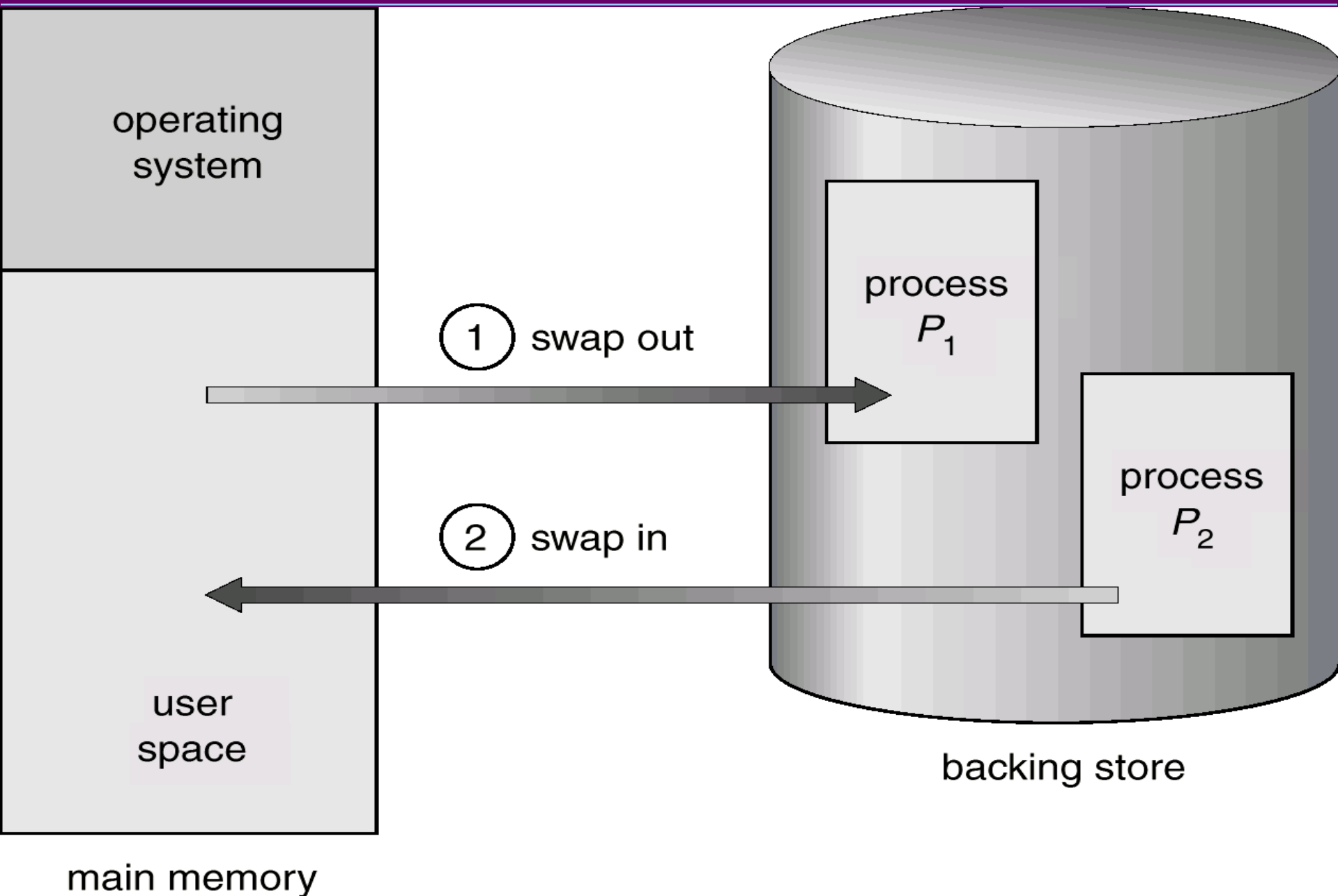


Swapping

- A process can be *swapped* temporarily out of memory to a *backing store*, and then brought back into memory for continued execution.
- Backing store – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images.

- *Roll out, roll in* – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed.
- Major part of swap time is transfer time; total transfer time is directly proportional to the *amount* of memory swapped.
- Modified versions of swapping are found on many systems, i.e., UNIX, Linux, and Windows.

Schematic View of Swapping



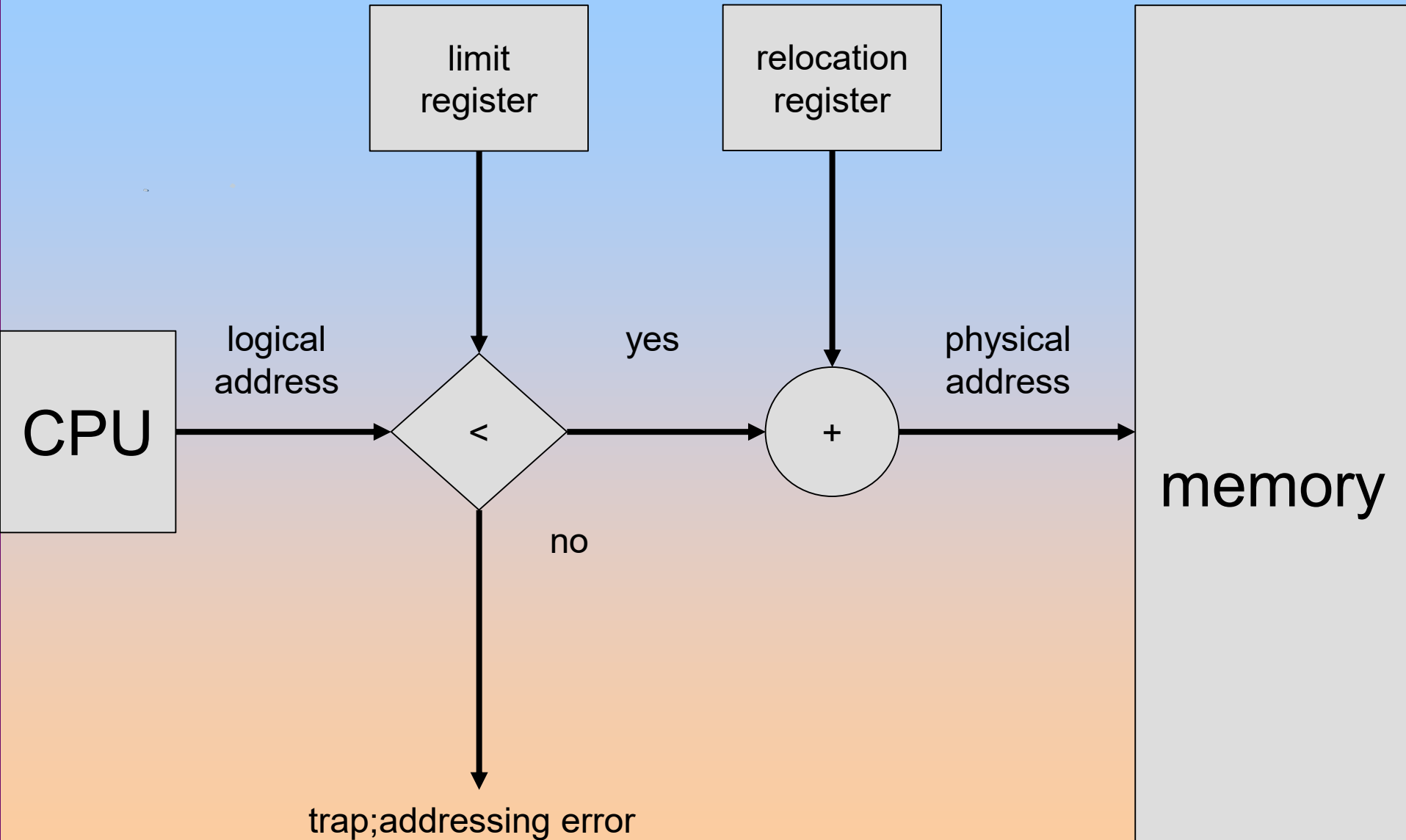
Contiguous Allocation

- Main memory usually into two partitions:
 - ☞ Resident operating system, usually held in low memory with interrupt vector.
 - ☞ User processes then held in high memory.

■ Single-partition allocation

- ☞ Relocation-register scheme used to protect user processes from each other, and from changing operating-system code and data.
- ☞ Relocation register contains value of smallest physical address; limit register contains range of logical addresses – each logical address must be less than the limit register.

Hardware Support for Relocation and Limit Registers



Contiguous Allocation (Cont.)

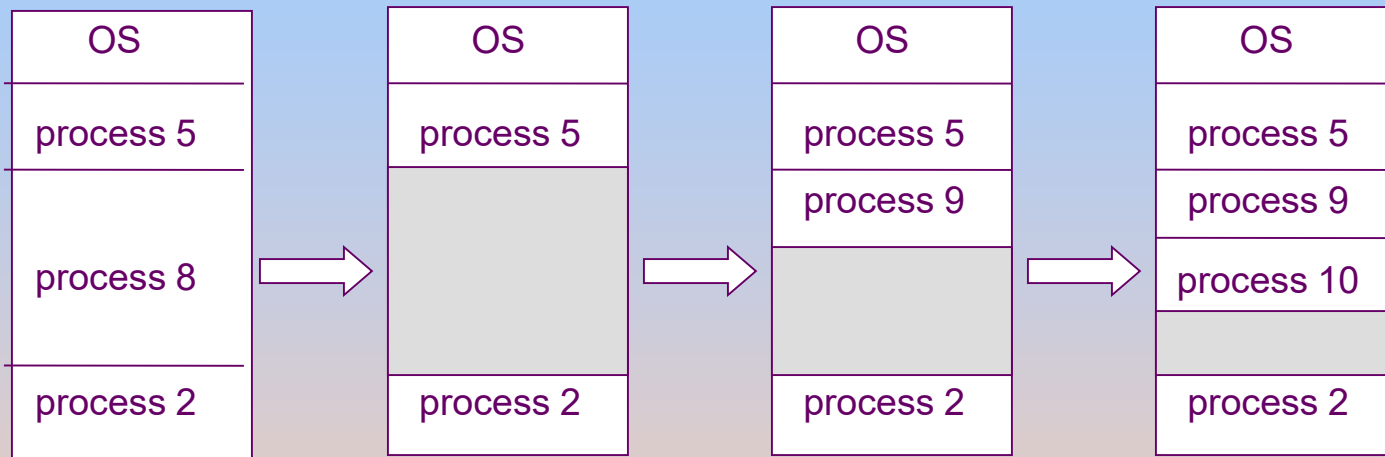
Multiple-partition allocation

Hole – block of available memory; holes of various size are scattered throughout memory.

When a process arrives, it is allocated memory from a hole large enough to accommodate it.

Operating system maintains information about:
a) allocated partitions b) free partitions (hole)

Contiguous Allocation (Cont.)



Dynamic Storage-Allocation Problem

How to satisfy a request of size n from a list of free holes.

- **First-fit:** Allocate the *first* hole that is big enough.
- **Best-fit:** Allocate the *smallest* hole that is big enough; must search entire list, unless ordered by size. Produces the smallest leftover hole.

- **Worst-fit:** Allocate the *largest* hole; must also search entire list. Produces the largest leftover hole.
- First-fit and best-fit better than worst-fit in terms of speed and storage utilization.

Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous.
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.

■ Reduce external fragmentation by compaction

- ➡ Shuffle memory contents to place all free memory together in one large block.
- ➡ Compaction is possible *only* if relocation is dynamic, and is done at execution time.

Paging

- Logical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available.
- Divide physical memory into fixed-sized blocks called **frames** (size is power of 2, between 512 bytes and 8192 bytes).
- Divide logical memory into blocks of same size called **pages**.

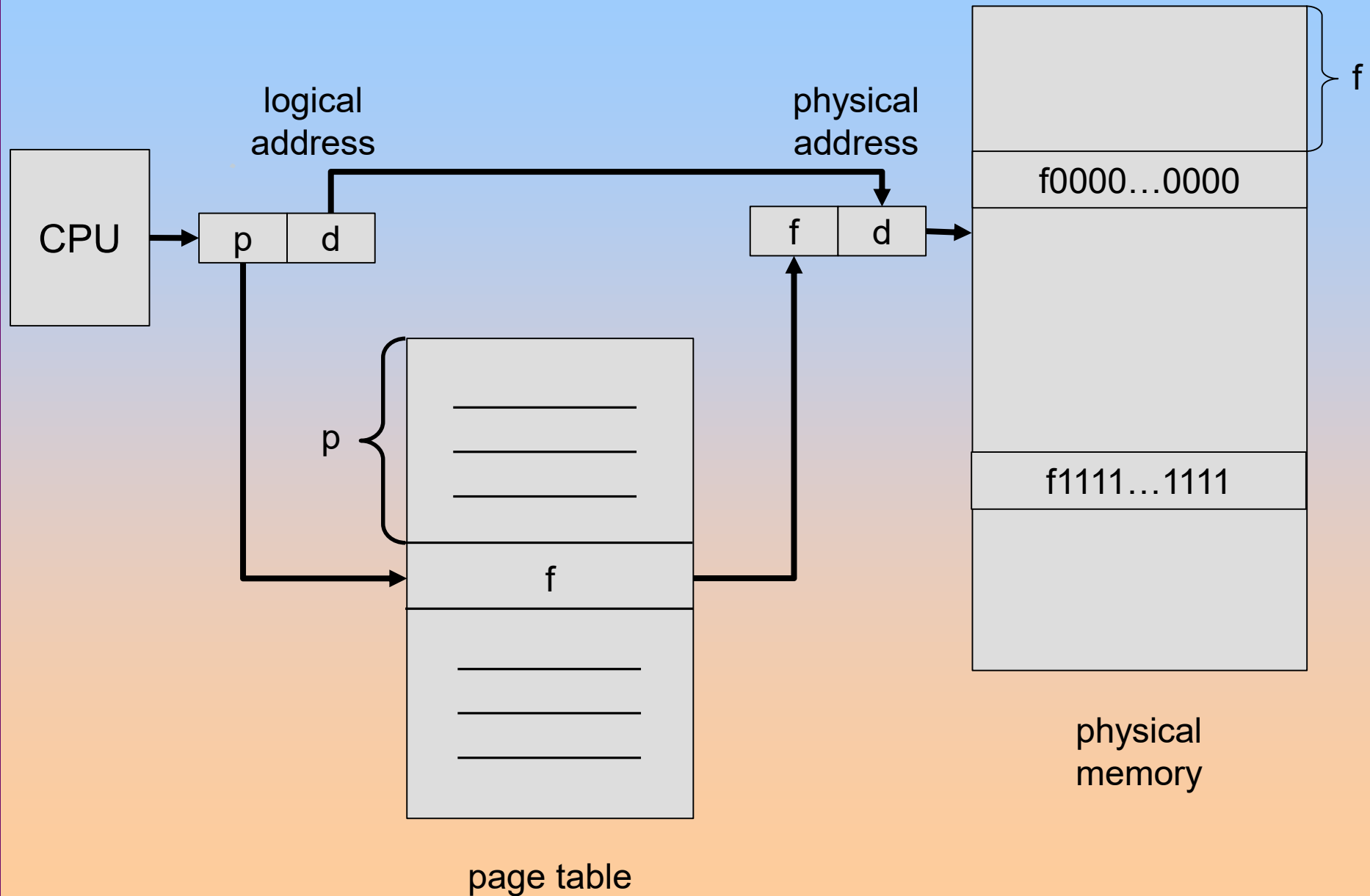
- Keep track of all free frames.
- To run a program of size n pages, need to find n free frames and load program.
- Set up a page table to translate logical to physical addresses.
- Internal fragmentation.

Address Translation Scheme

■ Address generated by CPU is divided into:

- ☞ *Page number (p)* – used as an index into a *page table* which contains base address of each page in physical memory.
- ☞ *Page offset (d)* – combined with base address to define the physical memory address that is sent to the memory unit.

Address Translation Architecture



Paging Example

page 0
page 1
page 2
page 3

logical
memory

0	1
1	4
2	3
3	7

page table

frame
number

0

1

2

3

4

5

6

7

page 0

page 2

page 1

page 3

physical
memory

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

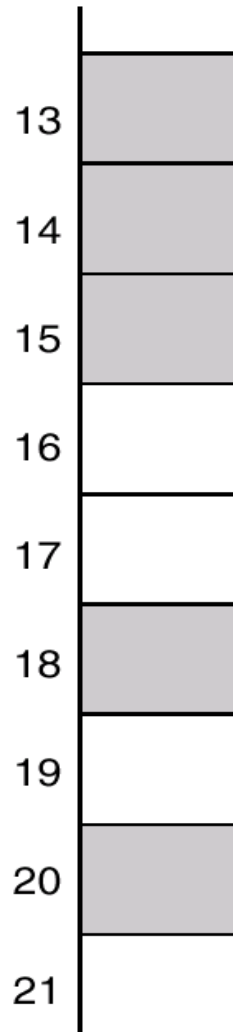
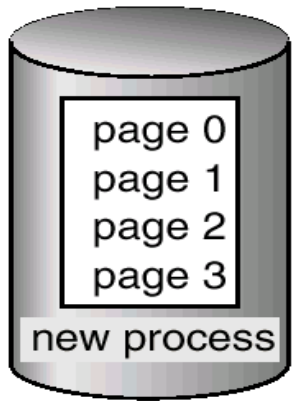
0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Free Frames

free-frame list

14
13
18
20
15

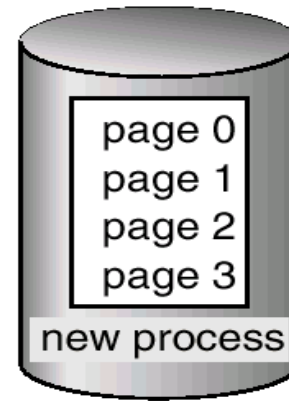


(a)

Before allocation

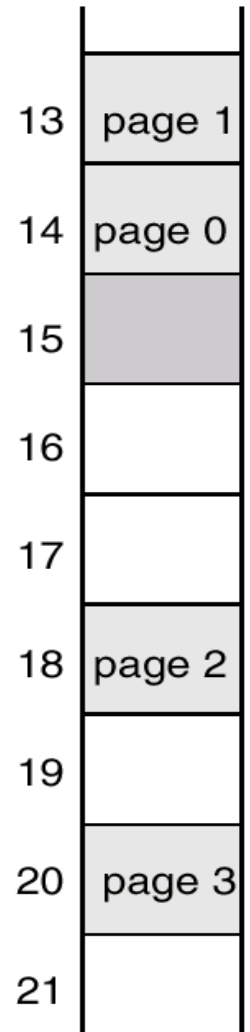
free-frame list

15



0	14
1	13
2	18
3	20

new-process page table



(b)

After allocation

Implementation of Page Table

- Page table is kept in main memory.
- *Page-table base register (PTBR)* points to the page table.
- *Page-table length register (PRLR)* indicates size of the page table.

- In this scheme every data/instruction access requires two memory accesses. One for the page table and one for the data/instruction.
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called *associative memory* or *translation look-aside buffers (TLBs)*

Associative Memory

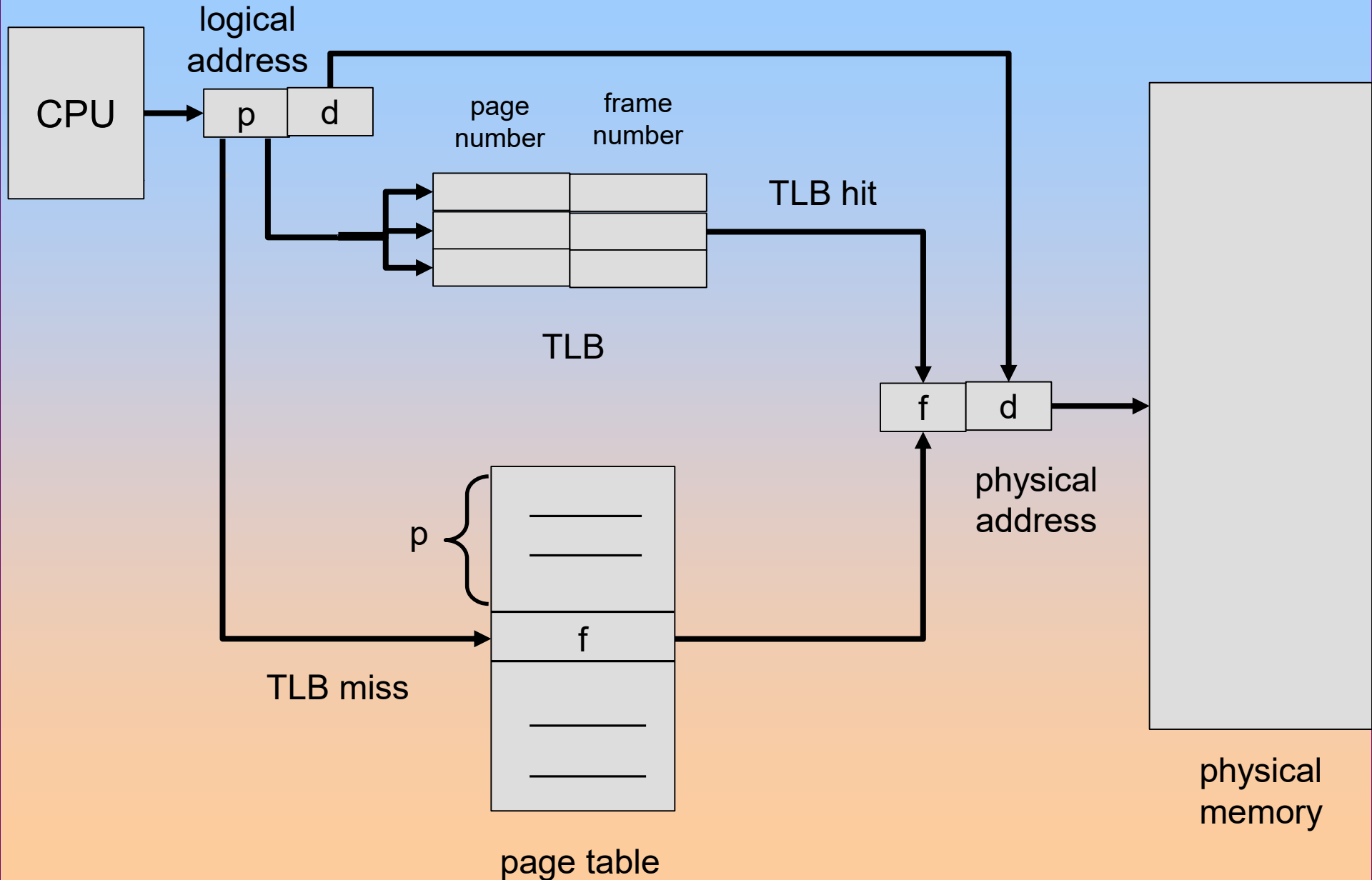
■ Associative memory – parallel search

Page #	Frame #

Address translation (A' , A'')

- ➡ If A' is in associative register, get frame # out.
- ➡ Otherwise get frame # from page table in memory

Address Translation Architecture



Effective Access Time

- Associative Lookup = ε time unit
- Assume memory cycle time is 1 microsecond
- Hit ratio – percentage of times that a page number is found in the associative registers; ratio related to number of associative registers.

- Hit ratio = α

- Effective Access Time (EAT)

$$\text{EAT} = (1 + \varepsilon) \alpha + (2 + \varepsilon)(1 - \alpha)$$

$$= 2 + \varepsilon - \alpha$$

Memory Protection

- Memory protection implemented by associating protection bit with each frame.
- *Valid-invalid* bit attached to each entry in the page table:

- ➡ “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page.
- ➡ “invalid” indicates that the page is not in the process’ logical address space.

Valid (v) or Invalid (i) Bit In A Page Table

00000	page 0
	page 1
	page 2
	page 3
	page 4
10,468	page 5
12,287	

frame number		valid—invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page <i>n</i>

Shared Pages

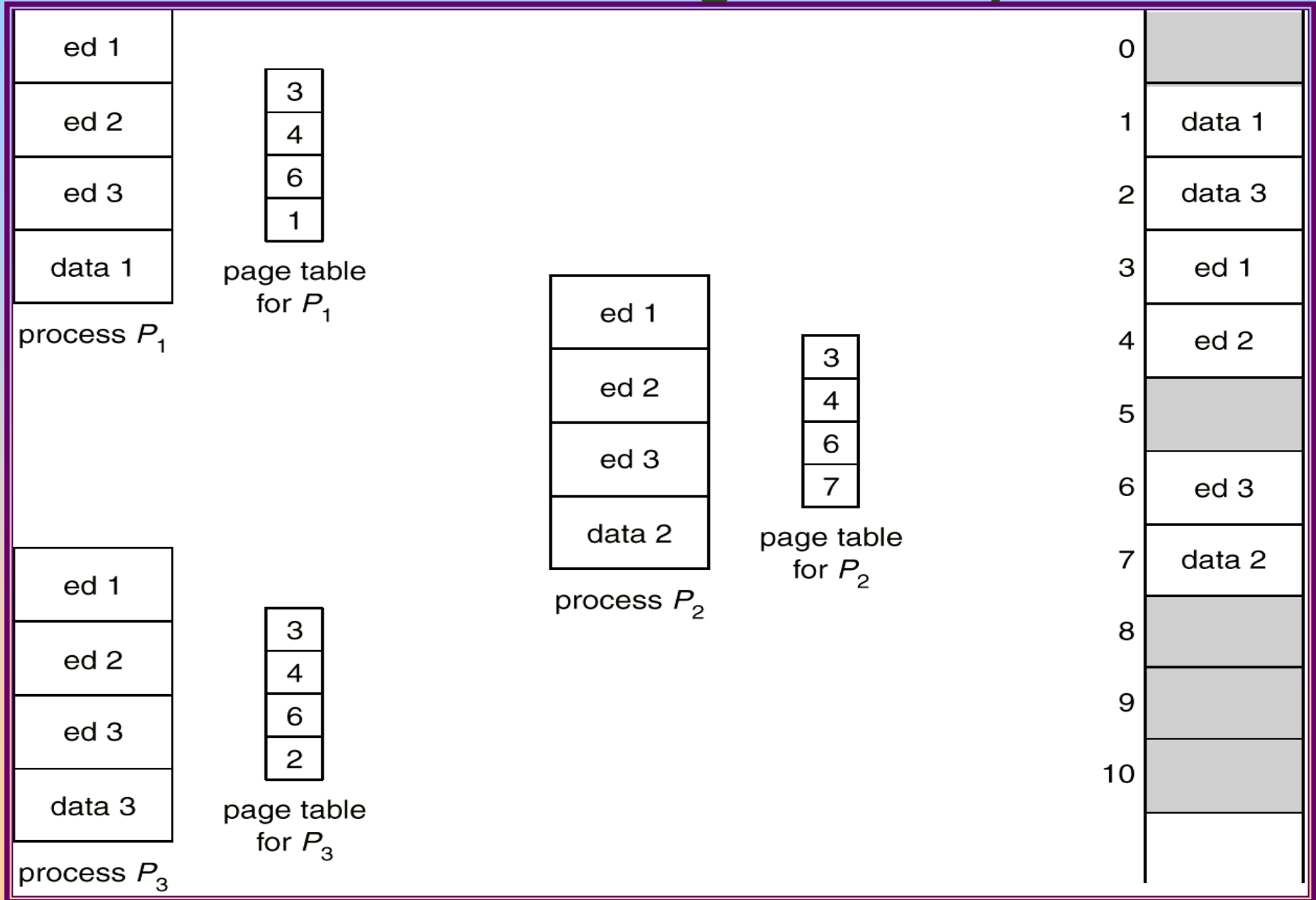
■ Shared code

- ➡ One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems).
- ➡ Shared code must appear in same location in the logical address space of all processes

■ Private code and data

- ➡ Each process keeps a separate copy of the code and data
- ➡ The pages for the private code and data can appear anywhere in the logical address space

Shared Pages Example



Page Table Structure

- Hierarchical Paging

- Hashed Page Tables

- Inverted Page Tables

Hierarchical Page Tables

- Break up the logical address space into multiple page tables.
- A simple technique is a two-level page table.

- A logical address (on 32-bit machine with 4K page size) is divided into:
 - ☞ a page number consisting of 20 bits.
 - ☞ a page offset consisting of 12 bits.
- Since the page table is paged, the page number is further divided into:
 - ☞ a 10-bit page number.
 - ☞ a 10-bit page offset.

Two-Level Paging Example

■ page number page offset

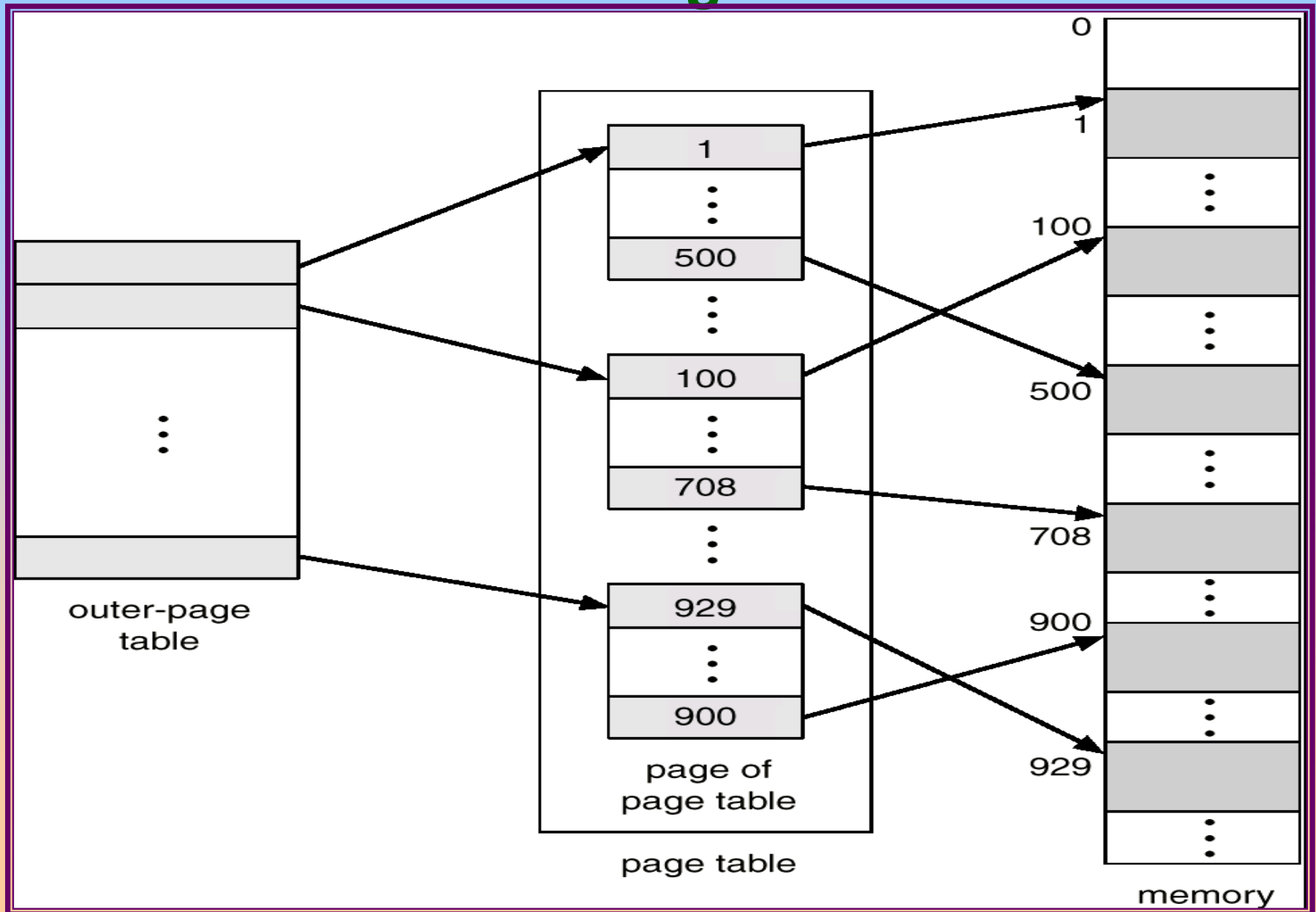


10

10

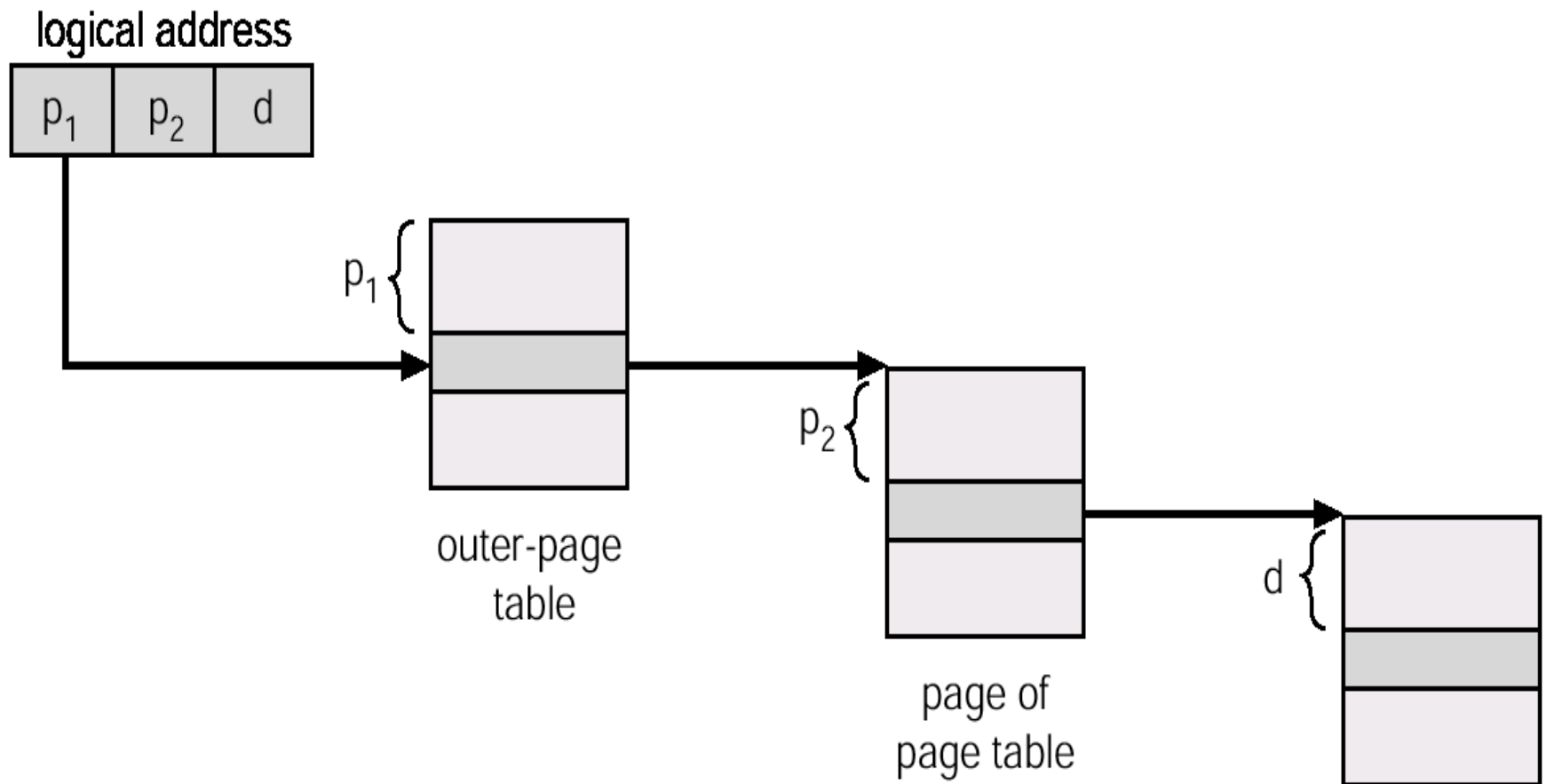
12

Two-Level Page-Table Scheme



Address-Translation Scheme

- Address-translation scheme for a two-level 32-bit paging architecture

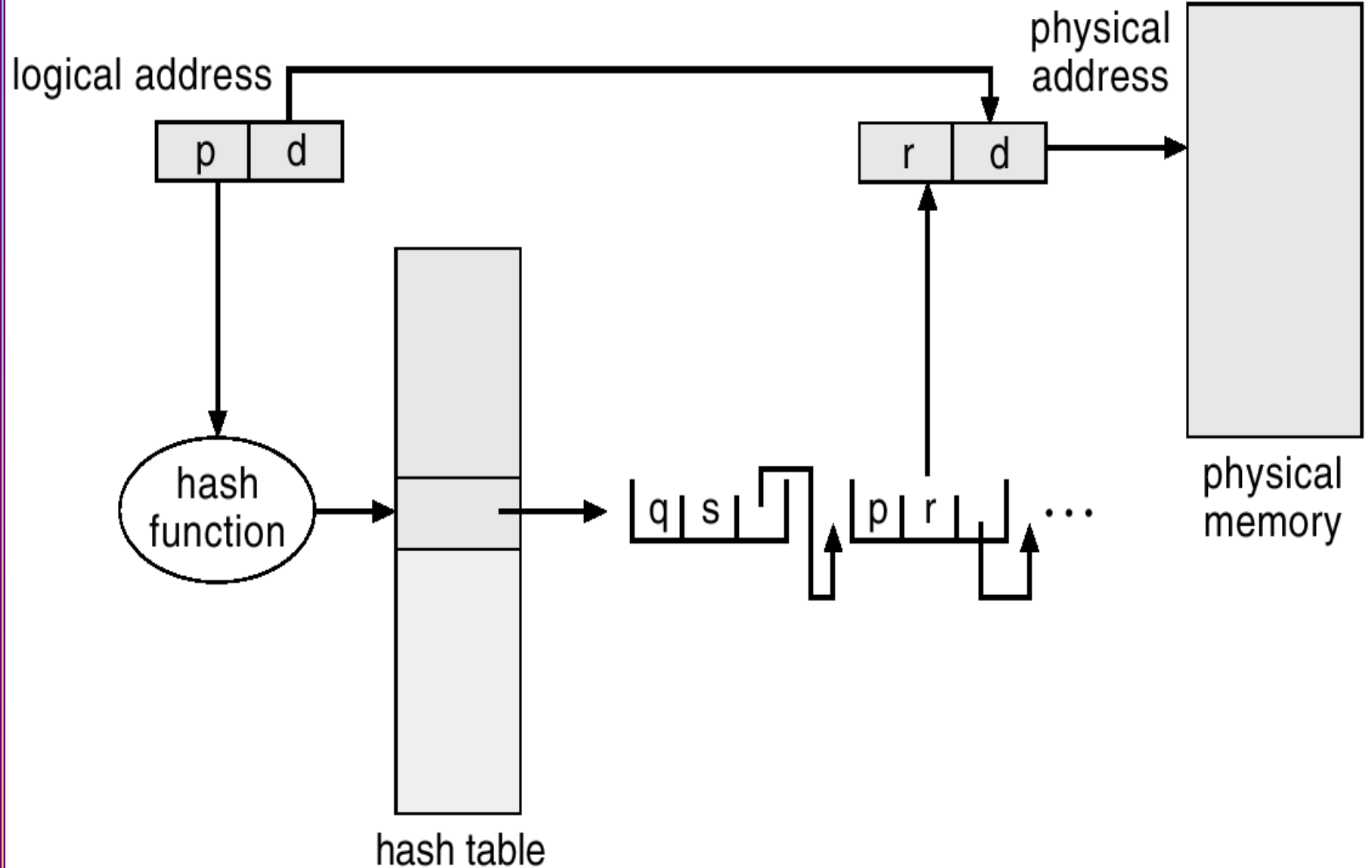


Hashed Page Tables

- Common in address spaces > 32 bits.
- The virtual page number is hashed into a page table. This page table contains a chain of elements hashing to the same location.

- Virtual page numbers are compared in this chain searching for a match. If a match is found, the corresponding physical frame is extracted.

Hashed Page Table

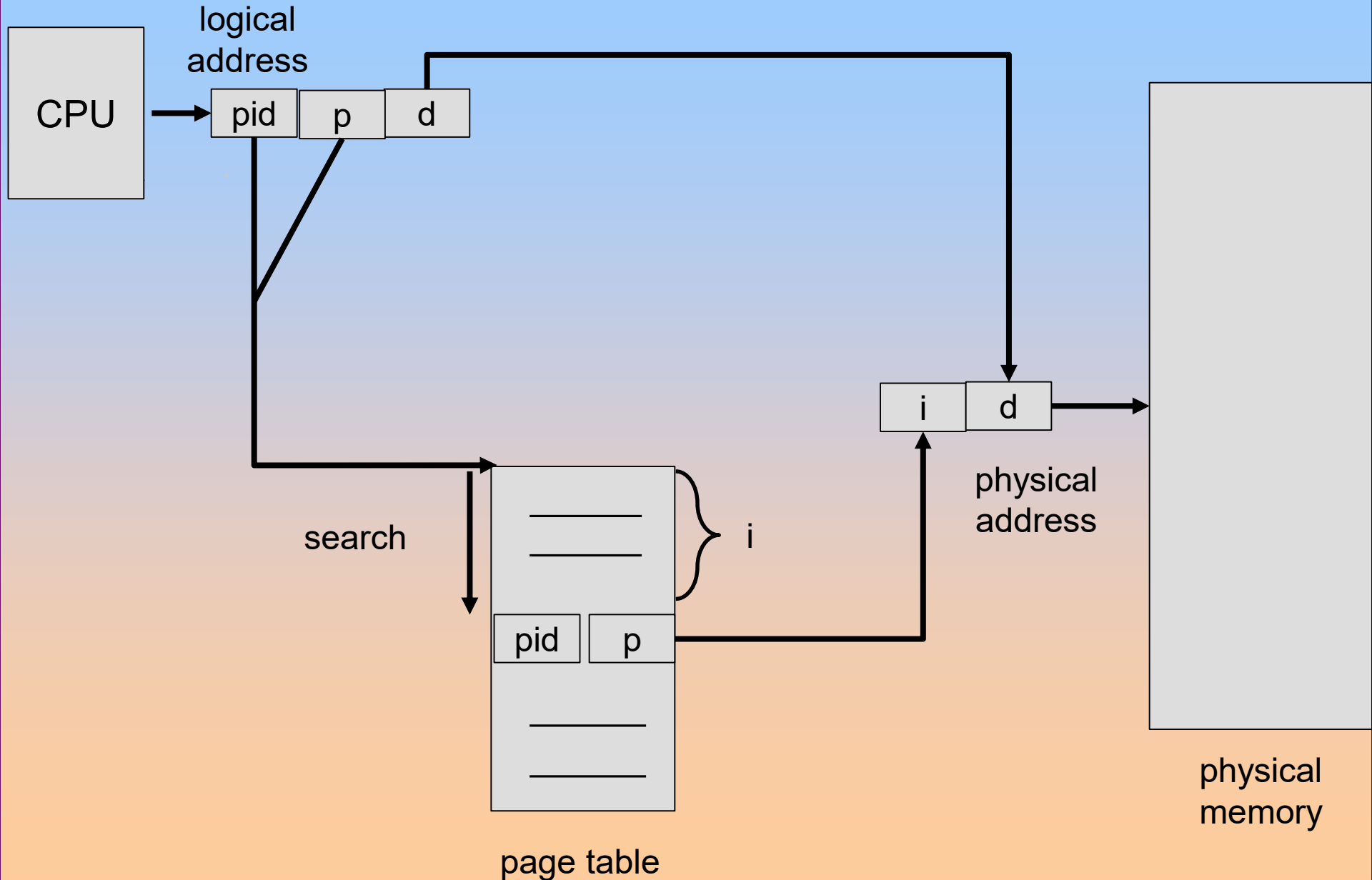


Inverted Page Table

- One entry for each real page of memory.
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page.

- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs.
- Use hash table to limit the search to one — or at most a few — page-table entries.

Inverted Page Table Architecture

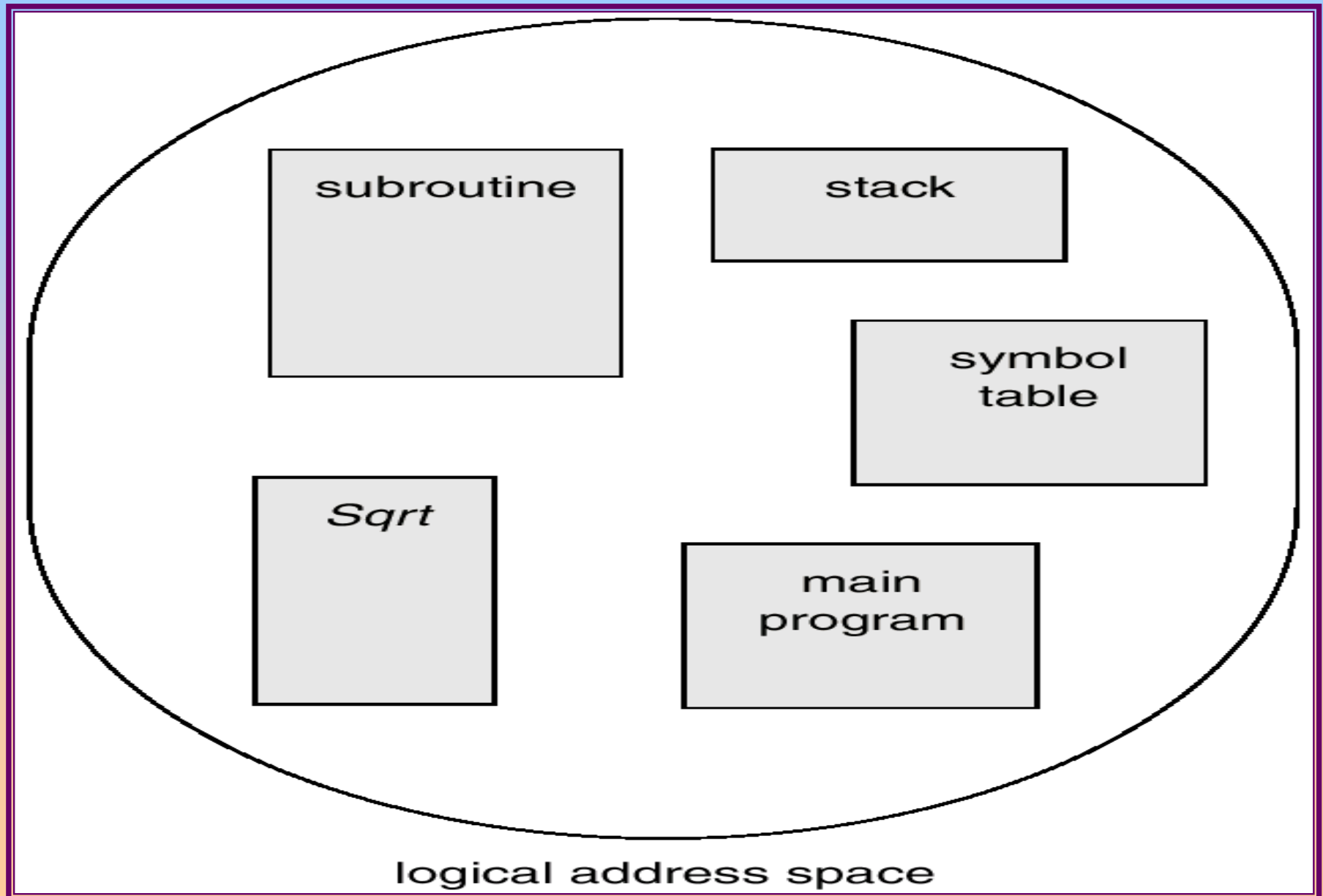


Segmentation

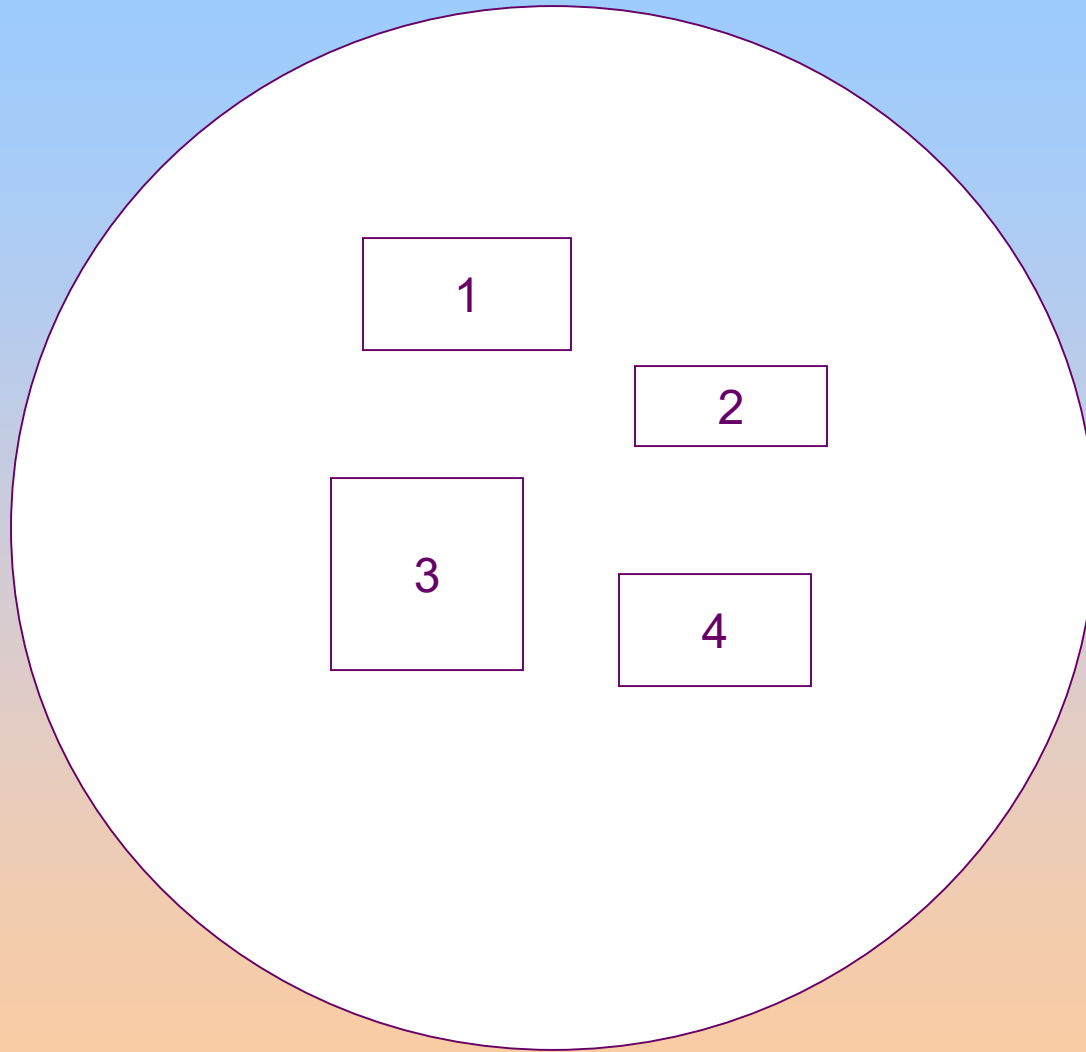
- Memory-management scheme that supports user view of memory.
- A program is a collection of segments.
A segment is a logical unit such as:

main program,
procedure,
function,
method,
object,
local variables, global variables,
common block,
stack,
symbol table, arrays

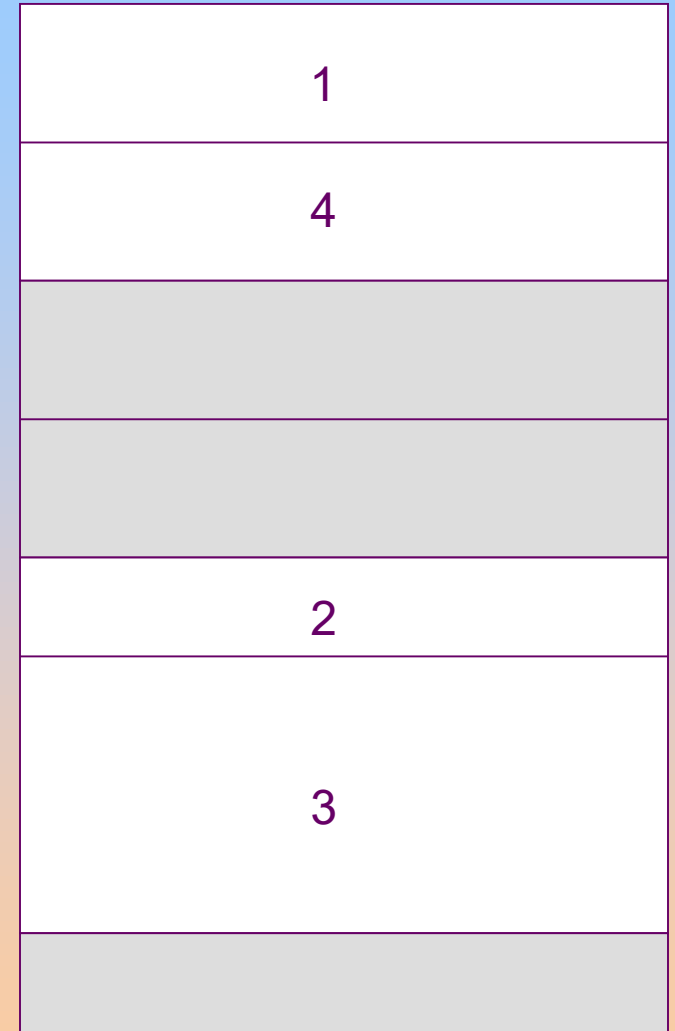
User's View of a Program



Logical View of Segmentation



user space



physical memory space

Segmentation Architecture

- Logical address consists of a two tuple:
 <segment-number, offset>,
- *Segment table* – maps two-dimensional physical addresses; each table entry has:
 - ☞ *base* – contains the starting physical address where the segments reside in memory.
 - ☞ *limit* – specifies the length of the segment.

- *Segment-table base register (STBR)* points to the segment table's location in memory.
- *Segment-table length register (STLR)* indicates number of segments used by a program;
segment number s is legal if $s < \text{STLR}$.

Segmentation Architecture (Cont.)

■ Relocation.

➡ dynamic

➡ by segment table

■ Sharing.

- ☞ shared segments
- ☞ same segment number

■ Allocation.

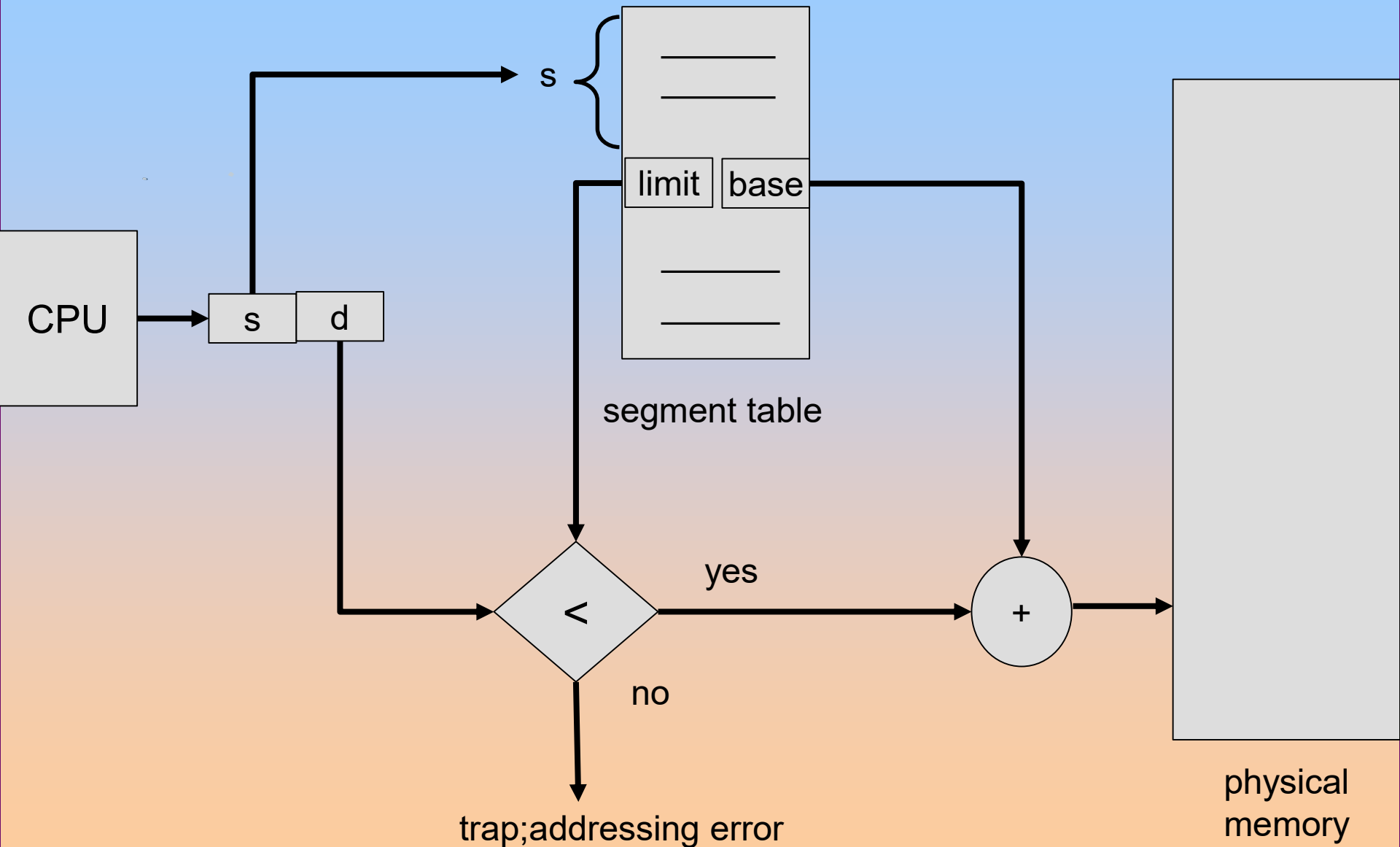
- ☞ first fit/best fit
- ☞ external fragmentation

Segmentation Architecture (Cont.)

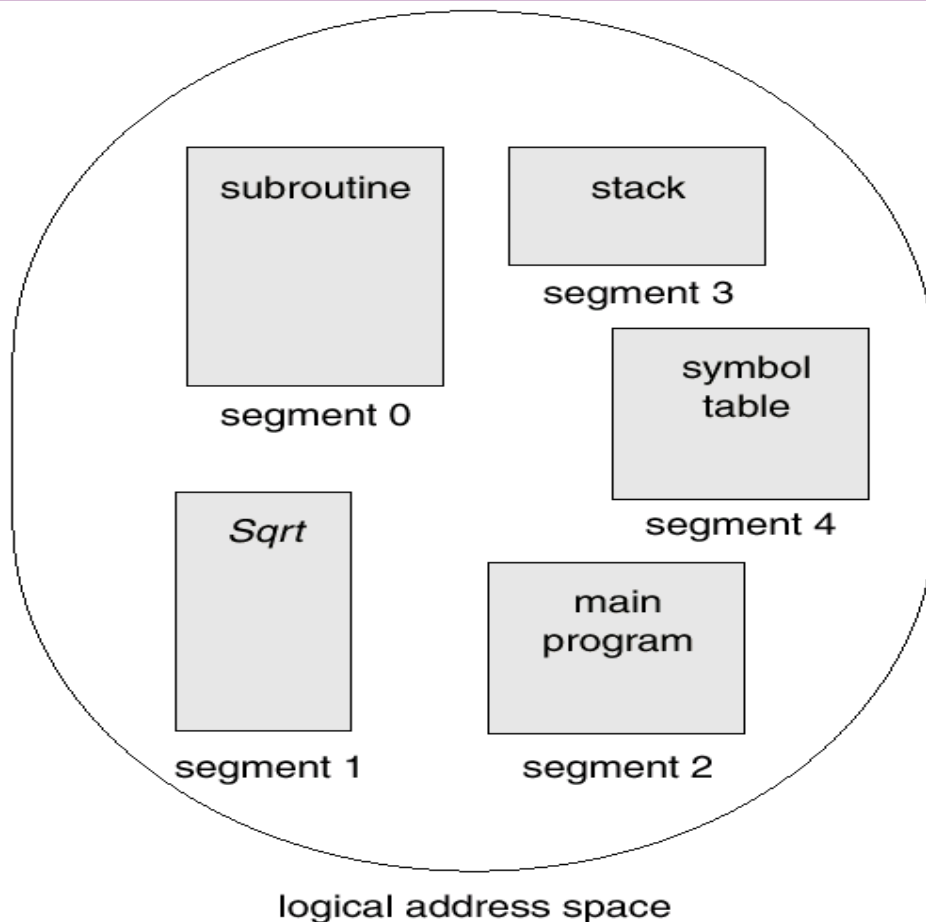
- Protection. With each entry in segment table associate:
 - ☞ validation bit = 0 $\Rightarrow$ illegal segment
 - ☞ read/write/execute privileges
- Protection bits associated with segments; code sharing occurs at segment level.

- Since segments vary in length, memory allocation is a dynamic storage-allocation problem.
- A segmentation example is shown in the following diagram

Segmentation Hardware

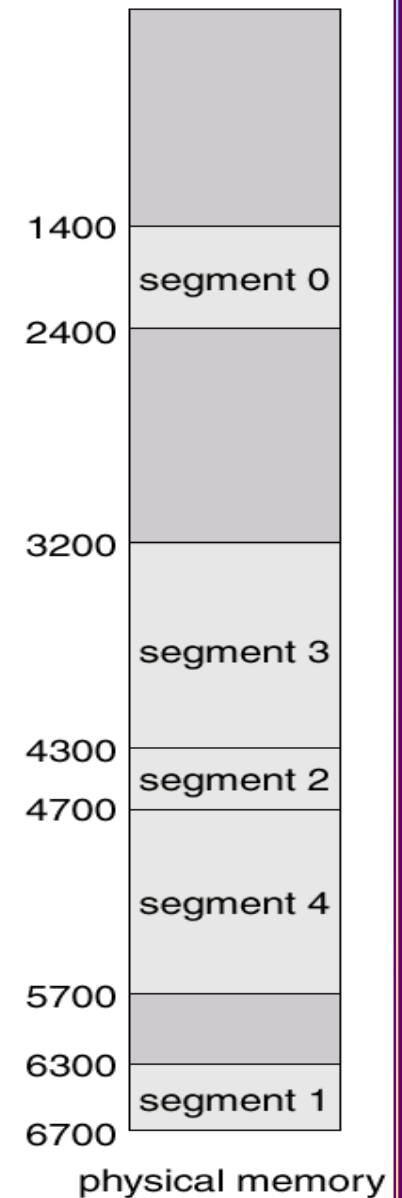


Example of Segmentation

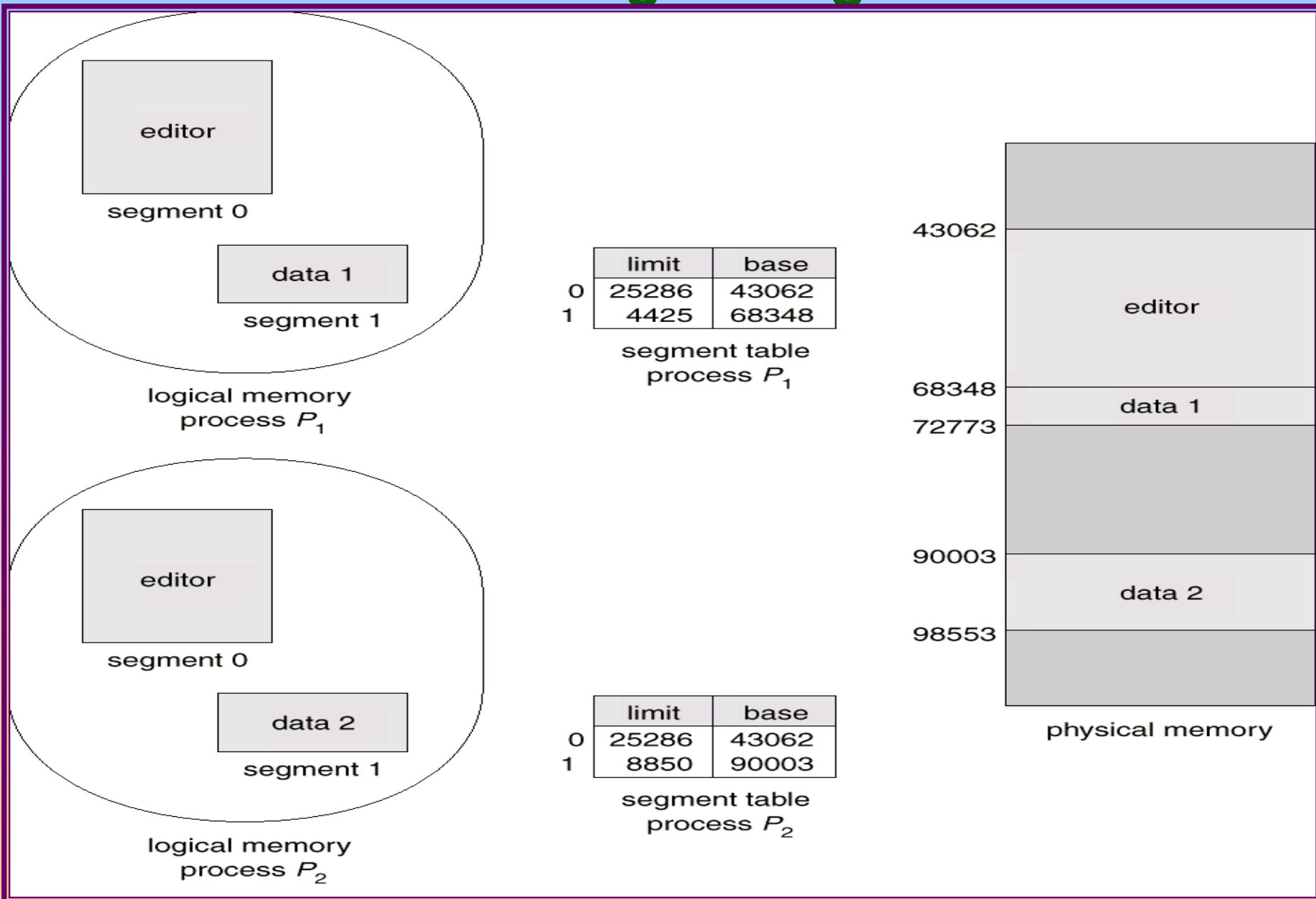


	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table



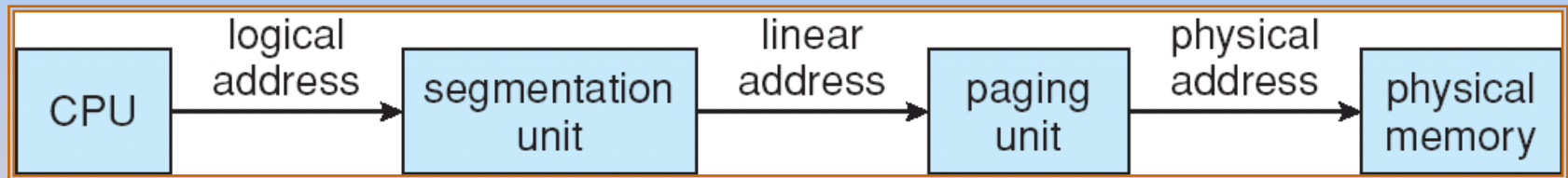
Sharing of Segments



Example: The Intel Pentium

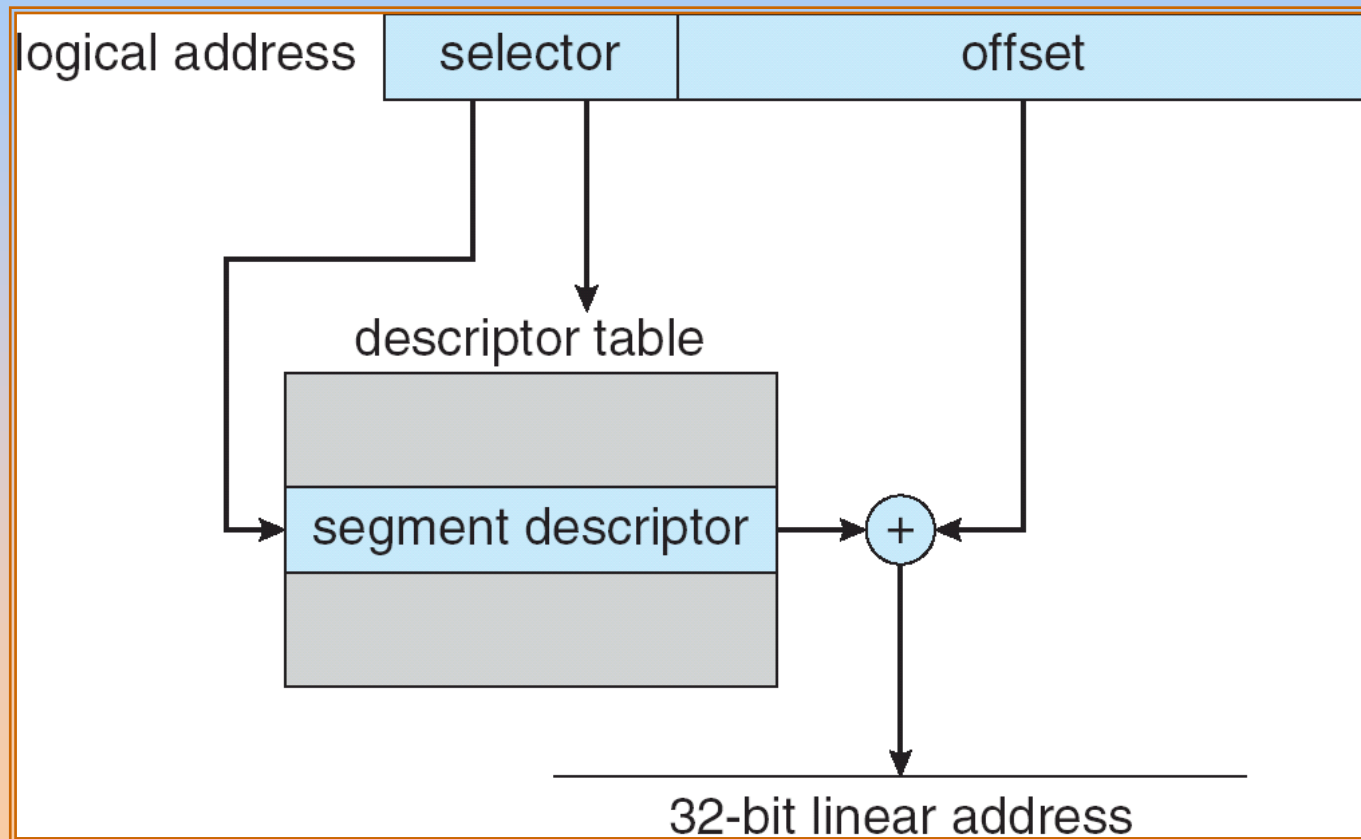
- Supports both segmentation and segmentation with paging
- CPU generates logical address
 - ➡ Given to segmentation unit
 - 📄 Which produces linear addresses
 - ➡ Linear address given to paging unit
 - 📄 Which generates physical address in main memory
 - 📄 Paging units form equivalent of MMU

Logical to Physical Address Translation in Pentium

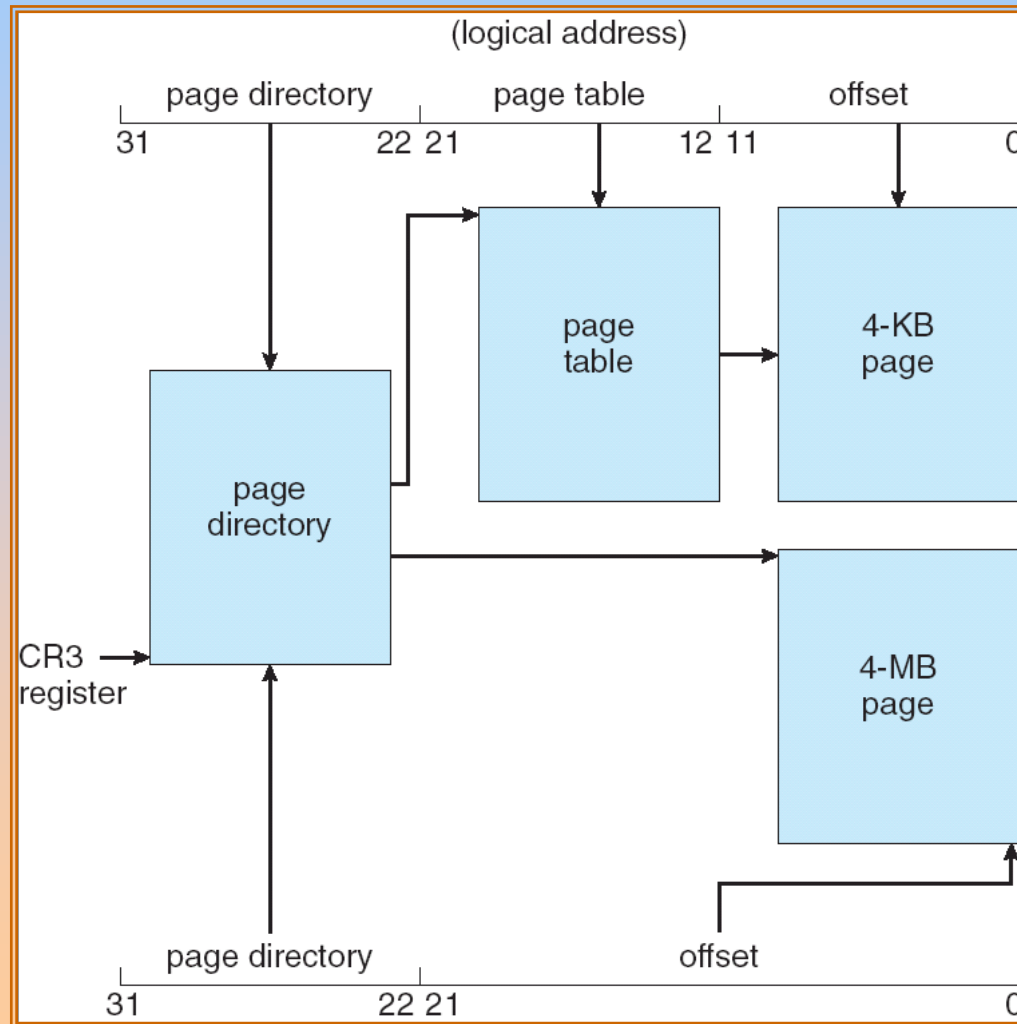


page number		page offset
p_1	p_2	d
10	10	12

Intel Pentium Segmentation



Pentium Paging Architecture



- 某计算机主存按字节编址，逻辑地址和物理地址都是**32** 位，页表项大小为**4**字节。请回答下列问题。
- 若使用一级页表的分页存储管理方式，逻辑地址结构为：
- 页号（**20**位） 页内偏移量（**12**位）
- 则该页的大小是多少字节？
- 该页表最大占用多少字节？

- (1) 因为页内偏移量是12位，所以页大小为4 KB。
- (2) 页表项数为 $2^{32}/4K=2^{20}$ ，该一级页表最大为 $2^{20} \times 4 \text{ B}=4 \text{ MB}$ 。

■ 分区分配内存管理方式的主要保护措施是

- A. 界地址保护
- B. 程序代码保护
- C. 数据保护
- D. 栈保护

■ A。

考查分区分配存储管理方式的保护措施。分区分配存储管理方式的保护措施是设置界地址寄存器。每个进程都有自己独立的进程空间，如果一个进程在运行时所产生的地址在其地址空间之外，则发生地址越界，即当程序要访问某个内存单元时，由硬件检查是否允许，如果允许则执行，否则产生地址越界中断，由操作系统进行相应处理。

■ 一个分段存储管理系统中，地址长度为32位，其中段号占8位，则最大段长是：

- A. 2^8B
- B. 2^{16}B
- C. 2^{24}B
- D. 2^{32}B

■ C。

考查分段存储管理系统，分页式存储系统也一样。段地址为**32**位二进制数，其中**8**位表示段号，则段内位移占用 $32 - 8 = 24$ 位二进制数，故最大段长为 2^{24}B 。

■ 某基于动态分区存储管理的计算机，其主存容量为**55MB**（初始为空闲），采用最佳适配（**Best Fit**）算法，分配和释放的顺序为：分配**15MB**，分配**30MB**，释放**15MB**，分配**8MB**，分配**6MB**，此时主存中最大空闲分区的大小是。

- **A. 7MB**
- **B. 9MB**
- **C. 10MB**
- **D. 15MB**

- **B。考查动态分区分配。**考生需对动态分区分配的四种算法加以理解。首次适配法(**first-fit**): 按分区在内存的先后次序从头查找, 找到符合要求的第一个分区进行分配。
- **最佳适配法(best-fit):** 按分区在内存的先后次序从头查找, 找到其大小与要求相差最小的空闲分区进行分配。**最坏适配法(worst-fit):** 按分区在内存的先后次序从头查找, 找到最大的空闲分区进行分配。

- Consider a paging system with the page table stored in memory.
- a. If a memory reference takes 300 time unit, how long does it take to access an instruction or data in a page that has been paged into memory?
- b. If we add TLB (translation look-aside buffers), and 80 percent of all page-table entries can be found in the TLB, what is the effective memory access time?
- (Assume that finding a page-table entry in the associative registers takes 20 time unit, if the entry is there.)

- $300 \times 2 = 600$

- $0.8 \times (300 + 20) + 0.2 \times (600 + 20) = 256 + 124 = 380$

- In a demand paging system, the page size is 1k bytes, and the page table is as follows(assuming use decimal values), translate a logical address 10 into its corresponding physical address, and why?

frame
2
1
6

- A. 8202 B. 4106 C. 2058
D. 1034